ANSIBLE EXPLANATION

Ansible ante **servers, applications, cloud infrastructure ni manual-ga configure cheyyakunda code/configuration files tho automate cheyyadaniki use chese tool**.
Simple-ga cheppalante:
“Nenu 100 servers meeda same work cheyyali. Okkokka server ki SSH chesi commands run cheyyadam badulu, Ansible tho okasari instructions rasukoni anni servers meeda execute cheyyagalanu.”
Ippudu zero nundi deep-ga chuddam.
1. Mundu problem enti?
Suppose neeku 1 Linux server undi.
Nuvvu application deploy cheyyali. Manual-ga:
ssh server1
sudo apt update
sudo apt install nginx
sudo systemctl start nginx
sudo systemctl enable nginx
1 server aithe manageable.
Kani company lo:
server1
server2
server3
...
server100
server500
Unnayi anuko.
Prathi server ki same commands manually execute cheyyadam:

- time-consuming
- mistakes vastayi
- configuration differences vastayi
- repeat cheyyadam kastam
- audit cheyyadam kastam

Appudu question:
“Ee server setup motham code laga describe chesi, automatic-ga execute cheyyalema?”
Yes.
That's where Ansible comes in.

---

2. Ansible actually em chestundi?
Ansible mainly **IT automation / configuration management / deployment / orchestration** kosam.
For example:

```
500 Linux Servers
    |
    v
  Ansible
    |
    +---- install packages
    +---- create users
    +---- modify configs
    +---- restart services
    +---- deploy applications
    +---- run commands
    +---- update servers
```

Nuvvu Ansible ki:
"All web servers lo Nginx install cheyyi, configuration copy cheyyi, service start cheyyi."
ani cheptav.
Ansible aa instructions ni target machines meeda execute chestundi.

---

3. Ansible ni enduku use chestaru?
Main reason:
Automation
Manual work ni automate cheyyadaniki.
Example:
Company lo 200 servers unnayi.
Every server lo:
Java install
Python install
Git install
Nginx install
User create
SSH configuration

Firewall configuration
Application deployment
Manual-ga cheyyadam huge work.
Ansible playbook lo once define chesthe:

```
- name: Install Git
  ansible.builtin.apt:
    name: git
    state: present
```

Ansible target servers meeda apply chestundi.

---

4. Ansible ni oka real-world example tho understand cheddam
Imagine school lo 100 students unnaru.
Principal cheptadu:
"Prathi student ki ID card ivvali."
Manual approach:
Student 1 -> ID card
Student 2 -> ID card
Student 3 -> ID card
...
Student 100 -> ID card
Automation approach:

```
Principal
  |
  v
Ansible
  |
  +--> Student 1
  +--> Student 2
  +--> Student 3
  ...
  +--> Student 100
```

Instruction:
"Prathi student ki ID card ivvu."
Ansible context lo:
"Prathi server lo Nginx install cheyyi."

---

5. Ansible architecture
Basic architecture ila untundi:

```
            Ansible Control Node
                  |
                  |
          ------------------------
          |       |       |
          v       v       v
        Server 1   Server 2   Server 3
```

Ansible usually **agentless**.
Ante target servers lo Ansible agent software continuously run avvalsina requirement generally undadu.
Linux/Unix targets ki Ansible commonly **SSH** use chestundi.
Windows targets ki commonly **WinRM/PSRP** mechanisms use chestundi.
So Linux example:

```
Ansible Controller
     |
     | SSH
     v
 Linux Server
```

---

6. Control Node ante enti?
Ansible run chese machine ni usually:
Control node
antaru.
Example:

```
Your laptop
   |
   +-- Ansible installed
or
AWS EC2
   |
   +-- Ansible installed
or
Jenkins server
   |
```

+-- Ansible
Control node nundi target machines ni manage chestav.

---

7. Managed Nodes ante enti?
Ansible manage chestunna machines:
Managed nodes
Example:
Control Node
|
+------ Web Server 1
+------ Web Server 2
+------ DB Server 1
+------ App Server 1
Ansible software target machine lo necessarily install cheyyalsina avasaram ledu for common Linux SSH-based management.
Idi Ansible yokka important concepts lo okati:
Agentless architecture

---

8. Inventory ante enti?
Ansible ki first question:
"Nenu ye servers ni manage cheyyali?"
Daaniki answer:
Inventory
Inventory lo servers list untayi.
Example:
[webservers]
web1.example.com
web2.example.com
web3.example.com

[dbservers]
db1.example.com
db2.example.com
Ikkada:
webservers
oka group.
dbservers
oka group.
Appudu nuvvu:
ansible webservers -m ping
ante:
webservers group lo unna machines ni contact cheyyi.

---

9. ansible **command enti?**
Ansible lo ad-hoc commands execute cheyyachu.
Example:
ansible all -m ping
Meaning:
all machines
|
v
ping/check connectivity
Output roughly:
server1 | SUCCESS
server2 | SUCCESS
server3 | SUCCESS

---

10. Module ante enti?
Idi **very important**.
Ansible lo actual work perform chese reusable units ni:
Modules
antaru.
Examples:
package module
service module
copy module
file module
user module
command module
shell module
Suppose Nginx install cheyyali.
Ubuntu/Debian:

```
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
```

Ikkada:
apt
module.
Module Ansible ki:
"Package install/manage cheyyali"
ani cheptundi.

---

11. Playbook ante enti?
Ansible automation ni YAML file lo write chestam.
Aa YAML file ni:
Playbook
antaru.
Example:

```
---
- name: Configure web servers
  hosts: webservers

  tasks:

    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
```

Idi Ansible world lo extremely important.
Think:

```
Playbook
  |
  +--- Play
        |
        +--- Task
        +--- Task
        +--- Task
```

---

12. Play ante enti?
Play ante basically:
Which machines meeda which tasks execute cheyyalo define chese unit.
Example:

```
- name: Configure web servers
  hosts: webservers
```

Ikkada:
name -> Configure web servers
hosts -> webservers
Ante webservers group meeda ee play execute avutundi.

---

13. Task ante enti?
Task ante:
One specific automation operation.
Example:

```
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
```

One task.
Another:

```
- name: Start nginx
  ansible.builtin.service:
    name: nginx
    state: started
```

Another task.
So:

```
Play
 |
```

+-- Task 1
+-- Task 2
+-- Task 3

---

14. YAML enduku?
Ansible configuration/playbooks mostly YAML lo rayachu.
YAML human-readable.
Example:
name: Install nginx
JSON tho compare chesthe:
{
"name": "Install nginx"
}
YAML much cleaner.
Anduke DevOps engineers ki readable ga untundi.

---

15. Idempotency — Ansible lo super important concept
Idi definitely understand cheskovali.
Suppose nuvvu cheppav:
"Nginx install cheyyi."
First time:
Nginx absent
↓
Ansible installs nginx
↓
changed
Second time same playbook run chesthe:
Nginx already installed
↓
Nothing to change
↓
ok
Ide:
Idempotency
Meaning:
Same desired state ni multiple times apply chesina, unnecessary changes repeatedly cheyyakunda desired state maintain cheyyadam.
Example:
state: present
Meaning:
package must exist.
Ansible check chestundi.
Already exists → nothing to do.

---

16. Desired State ante enti?
Automation tools lo important concept.
Nuvvu:
"Nginx install cheyyi"
ani imperative command laga think cheyyachu.
But Ansible mostly:
"Nginx should be installed."
ane desired state define chestundi.
Example:
state: present
Means:
Desired State:
Nginx = installed
Ansible current state check chesi desired state ki teesukeltundi.

---

17. Ansible actual flow
Let's say:
100 web servers
Nuvvu playbook run chestav:
ansible-playbook site.yml
Flow:

```
        Playbook
          |
          v
      Ansible Engine
          |
   ----------------------
```

```
   |     |     |
   v     v     v
 Server1 Server2 Server3
   |     |     |
   v     v     v
  Check state / execute modules
```

For example:
Server1 -> nginx installed? YES
Server2 -> nginx installed? NO
Server3 -> nginx installed? YES
Result:
Server1 -> ok
Server2 -> changed
Server3 -> ok
This ok / changed output Ansible learning lo important.

---

18. changed **ante enti?**
Ansible task execute chesi machine state change chesthe:
changed
Example:
Nginx wasn't installed
↓
Ansible installs it
↓
changed
Already installed unte:
Nginx already installed
↓
No modification
↓
ok

---

19. Variables ante enti?
Playbooks lo values dynamic-ga maintain cheyyadaniki variables use chestam.
Example:

```
app_port: 8080
app_name: myapp
```

Then:

```
- name: Configure application
  template:
    src: app.conf.j2
    dest: /etc/myapp/app.conf
```

Template lo:

```
port={{ app_port }}
```

Ansible variable value inject chestundi.
So same playbook ni different environments lo reuse cheyyachu.
Example:
DEV -> port 8080
QA -> port 8081
PROD -> port 80
Same automation structure, different variables.

---

20. Templates ante enti?
Configuration files dynamic-ga generate cheyyadaniki **Jinja2 templates** use chestam.
Example:

```
server {
  listen {{ app_port }};
  server_name {{ server_name }};
}
```

Variables:

```
app_port: 8080
server_name: example.com
```

Ansible render chesthe:

```
server {
  listen 8080;
  server_name example.com;
}
```

Very useful for configuration management.

---

21. Handlers ante enti?
Suppose config file change ayithe service restart cheyyali.

Instead of every time restart cheyyadam:
notify: Restart nginx
Then:
handlers:

```
 - name: Restart nginx
   ansible.builtin.service:
     name: nginx
     state: restarted
```

Flow:

```
Config changed?
    |
   YES
    |
    v
Notify handler
    |
    v
Restart nginx
```

Config change kakapothe:
No restart
This is another important Ansible pattern.

---

22. Roles ante enti?
Real projects lo playbooks huge aipovachu.
Example:
site.yml

```
 |
 +-- nginx setup
 +-- mysql setup
 +-- java setup
 +-- application deployment
 +-- monitoring
```

Everything one file lo pedite messy.
So Ansible has:
Roles
Role ante reusable, structured automation package.
Typical structure:

```
roles/
└── nginx/
    ├── tasks/
    │   └── main.yml
    ├── handlers/
    │   └── main.yml
    ├── templates/
    ├── files/
    ├── vars/
    ├── defaults/
    └── meta/
```

Example:

```
nginx role
   |
   +-- install nginx
   +-- copy config
   +-- start service
   +-- configure firewall
```

Then another project kuda same role reuse cheyyachu.

---

23. Ansible Vault ante enti?
Passwords/secrets playbook lo plain text ga pettadam dangerous.
Example:
db_password: MySecretPassword
Bad practice.
Ansible Vault use chesi sensitive data encrypt cheyyachu.
Conceptually:

```
secret.yml
    |
    v
Ansible Vault
    |
    v
```

Encrypted data
Then automation run time lo decrypt chesi use cheyyachu.

---

24. Ansible ekkada use chestaru?
Ansible use cases chala unnayi.
Server provisioning
Install OS packages
Create users
Configure SSH
Configure firewall
Configuration management
Nginx config
Apache config
Database config
Application config
System settings
Application deployment
Example:
Git
↓
Build
↓
Artifact
↓
Ansible
↓
Servers
↓
Application running
Infrastructure automation
Cloud environments lo:
AWS
Azure
GCP
resources tho automation workflows build cheyyachu.
Patch management
1000 servers unnayi.
Update packages
Restart services if needed
Verify status
Ansible tho automate cheyyachu.
User management
Create user
Add SSH key
Add sudo permissions
Remove old user

---

25. Ansible vs Shell Script
Idi common interview question.
Shell script:
apt install nginx
systemctl start nginx
Ansible:
```
- name: Install nginx
  apt:
    name: nginx
    state: present
```
Difference?
Shell script mostly:
"Ee commands execute cheyyi."
Ansible:
"Ee desired state maintain cheyyi."
Also Ansible:

- multiple machines manage cheyyadam easier
- YAML-based automation
- reusable modules
- idempotency
- inventory
- variables

- roles
- handlers
- secrets management
- structured output

provide chestundi.

---

26. Ansible vs Terraform

Idi kuda DevOps lo extremely important.

Simple distinction:

Terraform

Mostly:

Infrastructure create/manage cheyyadam

Example:

AWS VPC
EC2
Load Balancer
Security Group
RDS

Terraform:

"I need 3 EC2 instances."

Ansible

Mostly:

Those machines/services ni configure/manage cheyyadam

Example:

Install Java
Install Nginx
Deploy application
Modify config
Start service

So common workflow:

```
Terraform
  |
  | Create infrastructure
  v
AWS resources
  |
  v
Ansible
  |
  | Configure machines
  v
Applications
```

But real-world responsibilities can overlap depending on architecture.

---

27. Ansible vs Kubernetes

Also confuse avvakudadhu.

Kubernetes main purpose:

Containerized applications ni orchestrate cheyyadam.

Ansible main purpose:

General IT/infrastructure automation and configuration management.

Example:

```
Ansible
  ↓
Install/configure Kubernetes
```

Then:

```
Kubernetes
  ↓
Run/manage containers
```

Rendu different problems solve chestayi.

---

28. Ansible vs Jenkins

Jenkins is primarily:

CI/CD automation server.

Ansible is:

Automation/configuration/deployment tool.

Together use cheyyachu.

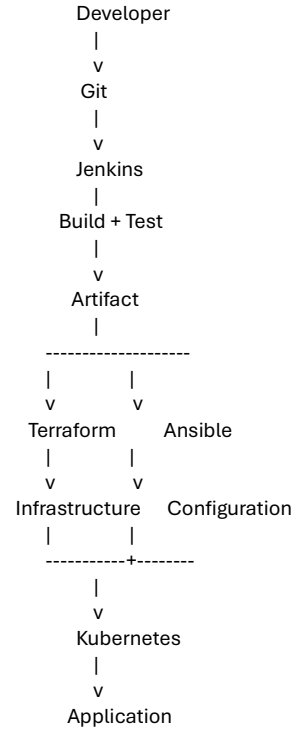
Example:

```
Developer pushes code
     ↓
   Jenkins
```

↓
Build + Test
↓
Ansible
↓
Deploy application
↓
Servers
So:
Jenkins = pipeline/orchestration platform
Ansible = automation/configuration tool
Though Jenkins can invoke many other tools too.

---

29. Complete DevOps picture
Ila imagine cheyyi:



Actual architectures vary, but this mental model useful.

---

30. Ansible lo important terminology
Ee terms definitely memorize cheyyi:

Term	Meaning
Control Node	Ansible run ayye machine
Managed Node	Ansible manage chese machine
Inventory	Servers list/groups
Module	Actual operation perform chese unit
Task	One automation operation
Play	Hosts + tasks collection
Playbook	YAML automation file
Variable	Dynamic value
Template	Dynamic configuration file
Handler	Change tarvata triggered task
Role	Reusable automation structure
Vault	Secret encryption
Idempotency	Repeated runs unnecessary changes avoid cheyyadam

---

31. Oka complete example
Suppose neeku:
3 web servers
Inventory:
[webservers]

web1
web2
web3
Playbook:

```
---
- name: Configure web servers
  hosts: webservers
  become: true

  tasks:

    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Run:
ansible-playbook webserver.yml
Ansible:
web1 -> install nginx -> start nginx
web2 -> install nginx -> start nginx
web3 -> install nginx -> start nginx
Output lo something like:
PLAY RECAP

web1 ok=2 changed=1 failed=0
web2 ok=2 changed=1 failed=0
web3 ok=2 changed=1 failed=0
Next time same playbook:
web1 ok=2 changed=0 failed=0
web2 ok=2 changed=0 failed=0
web3 ok=2 changed=0 failed=0
Enduku changed=0?
Because desired state already achieved.
That's the power of declarative/idempotent automation.

---

32. become: true **enti?**
Sometimes normal user ki administrative permissions undavu.
Example:
apt install nginx
requires root privileges.
Ansible lo:
become: true
use chesthe privilege escalation mechanism use chesi task ni elevated privileges tho execute cheyyachu.
Conceptually:

```
Normal user
   |
   | become
   v
Root privileges
   |
   v
Install package
```

---

33. Ansible learning order
Nuvvu Ansible serious-ga nerchukovali ante random topics jump avvakunda ee order follow avvu:

```
1. Linux basics
     ↓
2. SSH
     ↓
3. YAML
     ↓
4. Ansible installation
     ↓
5. Inventory
```

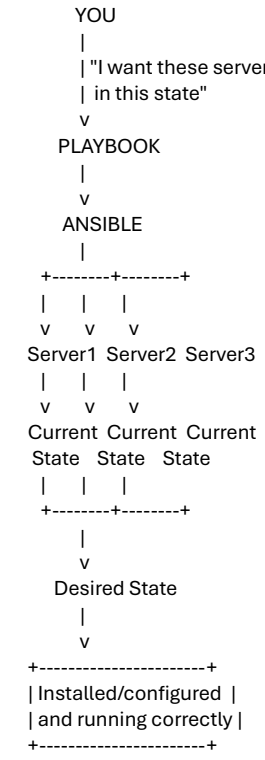
↓
6. Ad-hoc commands
↓
7. Modules
↓
8. Playbooks
↓
9. Variables
↓
10. Facts
↓
11. Conditionals
↓
12. Loops
↓
13. Handlers
↓
14. Templates/Jinja2
↓
15. Roles
↓
16. Vault
↓
17. Ansible Galaxy/collections
↓
18. Ansible with CI/CD
↓
19. Ansible with Cloud
↓
20. Production best practices

---

34. One sentence lo Ansible
If interviewer asks:
"What is Ansible?"
You can say:
Ansible is an agentless automation and configuration-management tool used to automate provisioning, configuration, application deployment, and orchestration across multiple systems using human-readable YAML playbooks.
Inka simple-ga:
“Servers ni manually configure cheyyakunda, code/configuration dwara automatically manage cheyyadaniki Ansible use chestam.”

---

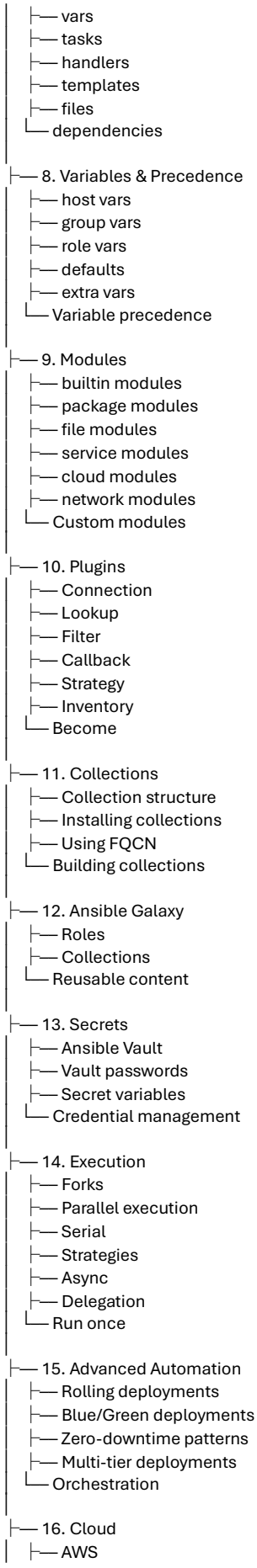
35. Finally, Ansible ni ila visualize cheyyi



Core idea: Ansible ante just "commands remote server lo run cheyyadam" kaadu. **Infrastructure/server configuration ni repeatable, consistent, version-controlled automation ga convert cheyyadam**.
Nuvvu DevOps nerchukuntunnav ante next logical step **"Ansible hands-on: laptop lo 2 Linux servers create chesi, inventory → SSH → first playbook → variables → loops → handlers → templates → roles"** ani end-to-end practical project cheyyadam. Adi chesthe concepts strong-ga settle avutayi.

"Total Ansible" ante inka chala depth undi. Nenu ippativaraku cheppindi roughly **foundation + intermediate + important advanced concepts**. Complete ecosystem ni cover cheyyalante inka layers unnayi.
Ansible ni total-ga map chesthe roughly ila untundi:

ANSIBLE
│
├── 1. Fundamentals
│ ├── Control Node
│ ├── Managed Nodes
│ ├── Agentless architecture
│ ├── SSH / WinRM
│ └── YAML
│
├── 2. Inventory
│ ├── Static inventory
│ ├── Dynamic inventory
│ ├── Groups
│ ├── Host variables
│ ├── Group variables
│ └── Inventory plugins
│
├── 3. Playbooks
│ ├── Plays
│ ├── Tasks
│ ├── Modules
│ ├── Variables
│ ├── Facts
│ ├── Loops
│ ├── Conditions
│ ├── Blocks
│ ├── Tags
│ ├── Register
│ └── Error handling
│
├── 4. Configuration Management
│ ├── Packages
│ ├── Users
│ ├── Files
│ ├── Permissions
│ ├── Services
│ ├── Cron
│ ├── SSH
│ └── System configuration
│
├── 5. Templates
│ ├── Jinja2
│ ├── Variables
│ ├── Filters
│ ├── Tests
│ ├── Conditionals
│ └── Loops
│
├── 6. Handlers
│ ├── notify
│ ├── listen
│ └── Service restart patterns
│
├── 7. Roles
│ ├── Role structure
│ ├── defaults


vars
tasks
handlers
templates
files
dependencies
8. Variables & Precedence
host vars
group vars
role vars
defaults
extra vars
Variable precedence
9. Modules
builtin modules
package modules
file modules
service modules
cloud modules
network modules
Custom modules
10. Plugins
Connection
Lookup
Filter
Callback
Strategy
Inventory
Become
11. Collections
Collection structure
Installing collections
Using FQCN
Building collections
12. Ansible Galaxy
Roles
Collections
Reusable content
13. Secrets
Ansible Vault
Vault passwords
Secret variables
Credential management
14. Execution
Forks
Parallel execution
Serial
Strategies
Async
Delegation
Run once
15. Advanced Automation
Rolling deployments
Blue/Green deployments
Zero-downtime patterns
Multi-tier deployments
Orchestration
16. Cloud
AWS

```
│   ├── Azure
│   ├── GCP
│   ├── Dynamic inventory
│   └── Cloud APIs
│
├── 17. Network Automation
│   ├── Cisco
│   ├── Juniper
│   ├── Arista
│   ├── Routers
│   ├── Switches
│   └── Network configuration
│
├── 18. Security & Compliance
│   ├── Hardening
│   ├── Patch management
│   ├── Compliance
│   ├── Remediation
│   └── Security automation
│
├── 19. Testing
│   ├── ansible-lint
│   ├── Molecule
│   ├── Unit testing
│   └── CI testing
│
├── 20. CI/CD
│   ├── Jenkins
│   ├── GitLab CI
│   ├── GitHub Actions
│   ├── Deployment pipelines
│   └── Automated testing
│
├── 21. Enterprise
│   ├── AWX
│   ├── Ansible Automation Platform
│   ├── RBAC
│   ├── Credentials
│   ├── Job Templates
│   ├── Workflows
│   ├── Scheduling
│   └── Audit / logging
│
└── 22. Production Engineering
    ├── Performance
    ├── Scalability
    ├── Security
    ├── Idempotency
    ├── Troubleshooting
    ├── Logging
    ├── Version control
    └── Best practices
```

So **“Ansible ante Playbook + YAML + SSH” ani anukunte chala small portion matrame.**

Oka important distinction

Ansible lo **concepts unlimited laga anipinchina**, DevOps job kosam anni same depth lo nerchukovalsina avasaram ledu.

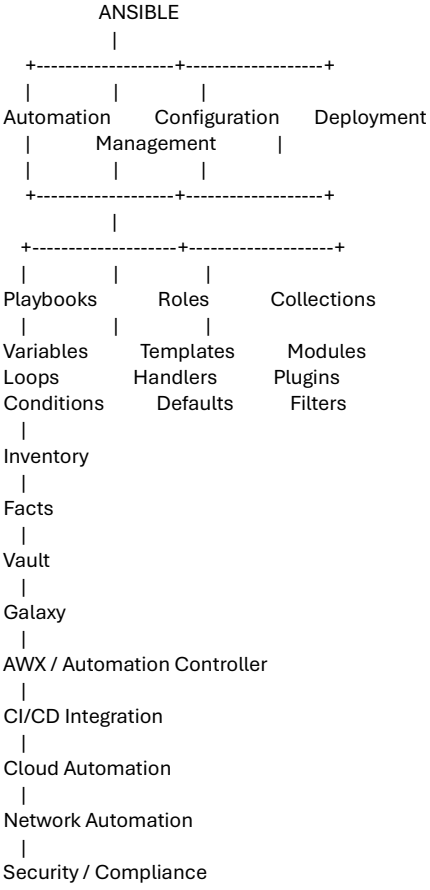
Usually strong foundation kosam:

Linux
↓
SSH
↓
YAML
↓
Inventory
↓
Modules
↓
Playbooks
↓
Variables

↓
Facts
↓
Loops
↓
Conditions
↓
Handlers
↓
Jinja2
↓
Roles
↓
Vault
↓
Collections
↓
CI/CD
↓
Cloud
↓
Production troubleshooting
Ivi solid-ga vaste, taruvata **AWX/Automation Platform, network automation, custom plugins/modules, advanced execution, enterprise architecture** laant

---

Ansible ante nenu paina cheppindi basic/core layer matrame. Ansible ecosystem chala pedda di. DevOps lo real projects ki vellinappudu inka chala concepts encounter avutayi.
Ansible ni oka **complete ecosystem** laga chuste clarity vastundi:

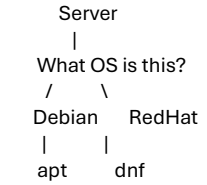


Ippudu **important advanced topics** enti automatic-ga information collect cheyy anedi chuddam.
1. Facts — server gurinchi information collect cheyyadam
Ansible target machine gurinchi automatic-ga information collect cheyyagaladu.
Example:
OS -> Ubuntu
Version -> 24.04
CPU -> 4 cores

Memory -> 16 GB
IP -> 10.x.x.x
Hostname -> web01
Architecture -> x86_64
Veetini **Ansible Facts** antaru.
Example:
```
- name: Show OS
  ansible.builtin.debug:
    var: ansible_facts.distribution
```
Output:
Ubuntu
Idi conditions lo chala useful.

---

2. Conditions — if/else
Example:
Ubuntu aithe apt use cheyyi, RedHat aithe dnf use cheyyi.
Concept:
when: ansible_facts.os_family == "Debian"
So:



Large environments lo idi important.

---

3. Loops
Suppose 5 packages install cheyyali.
Separate tasks rayakunda:
loop:
```
 - git
 - curl
 - vim
 - nginx
 - unzip
```
Ansible each item meeda operation perform chestundi.
git
curl
vim
nginx
unzip
↓
same task

---

4. Registered Variables
Task output ni variable lo store cheskovachu.
Concept:
register: result
Then:
debug:
var: result
Meaning:
Task execute
↓
Output
↓
Variable lo store
↓
Later tasks lo use
Automation logic build cheyyadaniki useful.

---

5. Blocks
Multiple tasks ni logical group ga organize cheyyachu.
Block
|
+-- Task 1
+-- Task 2
+-- Task 3
And error handling kuda implement cheyyachu:

block
 |
 +-- normal tasks
 |
rescue
 |
 +-- failure handling
 |
always
 |
 +-- cleanup
Programming lo:
try
catch
finally
concept laga think cheyyachu.

---

6. Tags
Large playbook lo particular tasks matrame run cheyyali anuko.
Example:
tags:
 - nginx
Then:
ansible-playbook site.yml --tags nginx
Only nginx-related tasks execute cheyyachu.
Useful when playbook huge ga untundi.

---

7. Ansible Check Mode
Production server ni modify cheyyakunda:
"Run chesthe em changes jarugutayo chupinchu."
ani check cheyyadaniki check mode use cheyyachu.
Concept:
ansible-playbook site.yml --check
So:
Actual execution
 ❌

Potential changes
 ↓
 SHOW
Production automation lo very useful.

---

8. Diff mode
Configuration change exactly enti ani inspect cheyyadaniki:
--diff
use cheyyachu.
Example:
Before:
port=80

After:
port=8080
Ila difference chudachu.

---

9. Serial deployment
Suppose 100 servers unnayi.
Anni okesari modify cheyyadam risk.
Instead:
100 servers

Batch 1 -> 10 servers
Batch 2 -> 10 servers
Batch 3 -> 10 servers
...
Ansible lo serial concept use cheyyachu.
Idi **rolling deployments** lo useful.

---

10. Delegation
Sometimes task target server meeda కాకుండా vere machine meeda execute cheyyali.
Example:
Web Server

|
| task
v
Load Balancer
Ansible lo delegation concepts unnayi.
Advanced orchestration lo important.

---

11. Async / Polling
Long-running task undi anuko.
Normal-ga task complete ayye varaku wait chestundi.
But certain operations asynchronous-ga start chesi later status check cheyyachu.
Start job
↓
Don't wait continuously
↓
Check later
↓
Completed?
Large automation workflows lo useful.

---

12. Ansible Plugins
Idi kuda big topic.
Modules tasks perform chestayi.
Plugins Ansible behavior ni extend/customize chestayi.
Examples:
Connection plugins
Callback plugins
Lookup plugins
Filter plugins
Inventory plugins
Strategy plugins
Example:
Lookup
↓
Fetch value from somewhere

Filter
↓
Transform data

Connection
↓
How to connect to target

---

13. Collections
Modern Ansible ecosystem lo **Collections** important.
Collection lo:
Modules
Plugins
Roles
Documentation
package cheyyachu.
Conceptually:
Collection
|
+-- modules
+-- plugins
+-- roles
+-- docs
Different vendors/projects their own collections provide cheyyachu.

---

14. Ansible Galaxy
Ansible Galaxy ante reusable Ansible content discover/share cheyyadaniki ecosystem.
For example someone already created:
nginx role
docker role
mysql role
apache role
Nuvvu from scratch create cheyyakunda appropriate reusable content ni use cheyyagalav.
Important distinction:
Galaxy

↓
Reusable Ansible content

---

15. Ansible Vault
Idi already mention chesam, but production lo important.
Secrets:
passwords
API keys
tokens
private data
encrypt cheyyadaniki Vault use chestaru.
Example conceptual flow:
secret
↓
Vault encryption
↓
encrypted file
↓
Ansible execution
↓
secret available when needed

---

16. Dynamic Inventory
Static inventory:
[web]
server1
server2
server3
Cloud environment lo servers dynamically create/delete avutayi.
Example:
AWS
|
+-- EC2
+-- EC2
+-- EC2
+-- EC2
Every time manually inventory update cheyyadam practical kaadu.
Dynamic inventory mechanisms cloud/API nundi hosts discover cheyyadaniki help chestayi.

---

17. Cloud Automation
Ansible servers configure cheyyadam matrame kaadu.
Cloud services tho kuda automation workflows build cheyyachu.
Example:
AWS
├── EC2
├── VPC
├── Security Groups
├── Load Balancers
└── Other services
Azure/GCP environments kuda ecosystem integrations/collections dwara automate cheyyachu.
But infrastructure lifecycle ki Terraform laanti tools ni kuda companies use chestayi; tool selection depends on the architecture.

---

18. Network Automation
Idi chala interesting area.
Ansible Linux servers kosame kaadu.
Network devices:
Cisco
Juniper
Arista
etc.
lanti environments lo configuration automation kosam use cheyyachu.
Example:
100 routers
|
v
Ansible
|
+-- VLAN config
+-- interface config
+-- routing config
+-- ACL config

Manual network configuration huge effort kabatti automation useful.

---

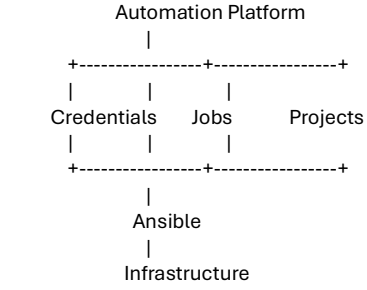
19. Security Automation
Ansible ni security workflows lo kuda use chestaru.
Example:
Server
 |
 +-- Disable unwanted service
 +-- Apply configuration
 +-- Update packages
 +-- Configure permissions
 +-- Enforce security settings
Compliance checks/remediation workflows kuda build cheyyachu.

---

20. CI/CD Integration
Idi DevOps engineers ki **very important**.
Example:
Developer
  |
  v
Git
  |
  v
CI Pipeline
  |
  +-- Build
  +-- Test
  |
  v
Ansible
  |
  +-- Deploy
  +-- Configure
  +-- Verify
  |
  v
Production
Example:
ansible-playbook deploy.yml
pipeline lo run cheyyachu.

---

21. Ansible Automation Platform / AWX
CLI lo:
ansible-playbook
run cheyyadam small environments ki okay.
Large organization lo:
100s/1000s of servers
many teams
many playbooks
RBAC
credentials
audit
scheduling
job history
lanti requirements vastayi.
Appudu **Ansible Automation Platform** ecosystem / related automation-controller technology important avutundi.
Open-source side lo **AWX** project kuda relevant.
Conceptually:



---

22. Strategy Plugins

Suppose 100 servers unnayi.
Tasks ela execute avvali?
Server1
Server2
Server3
...
parallelism, execution strategy etc. control cheyyadaniki strategies important.
Common concept:
linear
vs other execution approaches.
Advanced Ansible performance understanding lo idi vastundi.

---

23. Ansible Performance
Production lo:
10 servers
manage cheyyadam easy.
But:
10,000 servers
manage cheyyali ante:

- forks
- parallelism
- fact gathering
- connection reuse
- execution strategy
- inventory performance
- task design

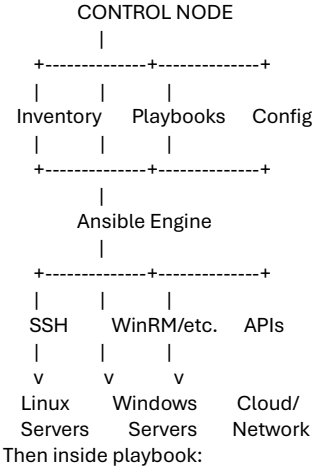
lanti topics important avutayi.
Appudu Ansible ni just YAML tool laga chuste saripodu.

---

24. Testing Ansible code
Real production automation lo:
"Playbook rasanu → direct production lo run chesa"
ane approach dangerous.
Ansible automation ni test cheyyadaniki tooling/practices unnayi.
Common ecosystem:
ansible-lint
Molecule
CI pipelines
Concept:
Write playbook
↓
Lint
↓
Test
↓
CI
↓
Deploy

---

25. Ansible architecture ni deeper-ga chuste
High level:



Then inside playbook:

Playbook
 |
 +-- Play
  |
  +-- Tasks
  | |
  | +-- Modules
  |
  +-- Variables
  +-- Conditions
  +-- Loops
  +-- Handlers
  +-- Templates
And reusable layer:
Roles
Collections
Galaxy
Vault
Plugins

---

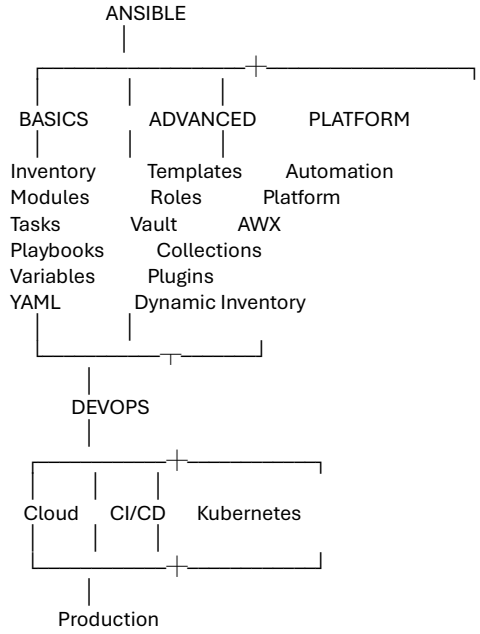
26. So Ansible = just server automation aa?
No.
Idi important.
Ansible ni broad-ga:
Automation engine/ecosystem
ga think cheyyi.
It can participate in:
Server automation
Configuration management
Application deployment
Cloud automation
Network automation
Security automation
Orchestration
CI/CD
Compliance/remediation
Infrastructure operations

---

27. But Ansible nerchukune person ki first day anni avasaram ledu
Ikkada biggest mistake beginners chestaru.
"Ansible lo 30 concepts unnayi → anni first learn cheyyali."
No.
First ee core path:
Linux
 ↓
SSH
 ↓
YAML
 ↓
Inventory
 ↓
Modules
 ↓
Playbooks
 ↓
Variables
 ↓
Loops
 ↓
Conditions
 ↓
Handlers
 ↓
Templates/Jinja2
 ↓
Roles
Then:
Vault
Facts
Collections
Galaxy

Dynamic Inventory
Testing
CI/CD
Cloud
AWX / Automation Platform
Performance

One big picture
Ansible ni oka **building** laga imagine cheyyi:



So **yes, Ansible lo inka chala untayi**. Nenu mundu explain chesindi **foundation** మాత్రమే.
Nuvvu really Ansible nerchukovali ante next step **theory ni inka expand cheyyadam kante practical-ga oka mini infrastructure build cheyyadam**. For example, 2 Linux servers → SSH → inventory → ad-hoc → first playbook → variables → loops → conditions → handlers → Jinja2 template → role → Vault → deployment ani one project complete chesthe Ansible almost entire flow mind lo settle avtundi.

Ansible
├— Core
├— Inventory
├— Modules
├— Playbooks
├— Variables & Facts
├— Conditions & Loops
├— Handlers
├— Jinja2 / Templates
├— Roles
├— Collections / Galaxy
├— Plugins
├— Vault / Security
├— Dynamic Inventory
├— Execution / Strategies
├— Cloud
├— Network Automation
├— CI/CD
├— Testing
├— Production Deployment
└— AWX / Ansible Automation Platform

Absolutely. Ippudu **Ansible ni complete picture ga, beginner nundi production/enterprise level varaku** explain chestha. Current official Ansible docs ni base cheskuntunna; terminology/version-related points lo current docs ni follow chesthunna. **A** Ansible Documentation+1

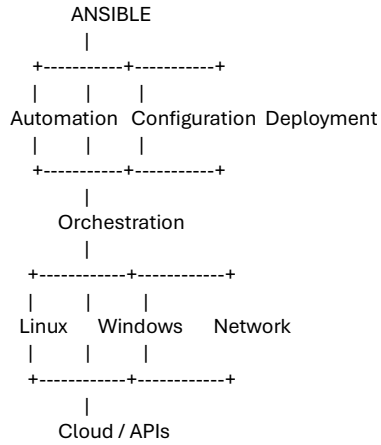
1. First: Ansible ni exact-ga define cheddam

Ansible = remote systems ni automate cheyyadaniki, configure cheyyadaniki, applications deploy cheyyadaniki, multiple machines madhya workflows orchestrate cheyyadaniki use chese automation framework.

Official docs prakaram Ansible:

- remote systems management automate chestundi
- desired state maintain chestundi
- agentless architecture use chestundi
- YAML playbooks use chestundi
- idempotent automation ni support chestundi. **A** Ansible Documentation+1

Big picture:



---

2. Ansible ecosystem lo first distinction

Chala mandi ikkada confuse avutaru.

Ansible ane word sometimes complete product/ecosystem ni refer chestundi.

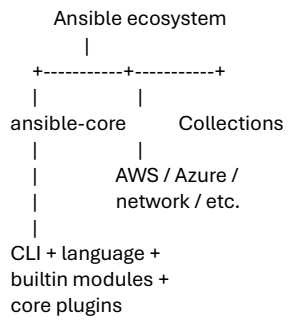
ansible-core ante actual core engine/framework.

ansible-core lo:

- CLI tools
- Ansible language
- built-in modules
- core plugins
- collection architecture

untayi. Current docs ansible-core ni Ansible ki main building block ani describe chestunnayi. **A** Ansible Documentation

So:



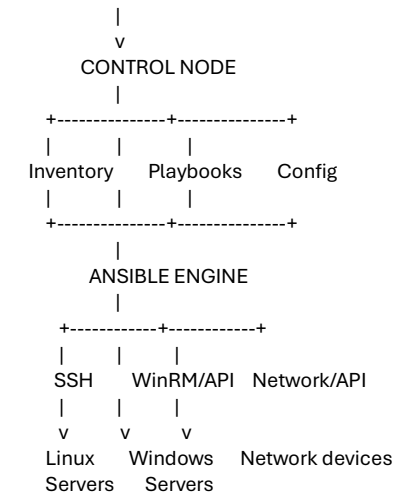
Idi first important distinction.

---

3. Complete architecture

Ippudu actual architecture chuddam.

YOU



Ansible control node nundi target machines ni manage chestundi. Managed nodes lo normally Ansible install cheyyalsina avasaram ledu. **A** Ansible Documentation

---

4. Control Node

Control node = Ansible commands/playbooks run chese machine.

Example:

Laptop
  |
  +-- ansible
  +-- ansible-playbook
  +-- ansible-inventory
  +-- ansible-vault

Leda:

Jenkins server
    |
    +-- Ansible

Leda:

Automation server
    |
    +-- Ansible

Current docs prakaram laptop, server, shared machine, container-based execution environment kuda control node ga use cheyyachu. **A** Ansible Documentation

---

5. Managed Node

Ansible control chese target:

web01
web02
db01

ivi **managed nodes** / **hosts**.

Example:

Control Node
   |
   +---- SSH ----> web01
   +---- SSH ----> web02
   +---- SSH ----> db01

Linux target lo generally:

Ansible installed? NO
SSH available? YES
Python/runtime requirements? Depends on module/environment

Ansible agent-based model kaadu. **A** Ansible Documentation

---

6. Inventory — Ansible ki "ye machines?"

Inventory ni simple-ga:

Ansible manage cheyyalsina machines database/list
ani think cheyyi.
Example:
[webservers]
web01
web02
web03

[dbservers]
db01
db02
Ippudu Ansible ki:
webservers = web01, web02, web03
dbservers = db01, db02
ani telustundi.
Inventory groups valla:
ansible webservers ...
ani target cheyyachu.
Official docs prakaram inventory hosts ni list cheyyadam tho paatu grouping and variables assignment ki kuda use avtundi. **A**


Ansible Documentation

---

7. Inventory deeper level
Inventory lo host variables:
[webservers]
web01 ansible_host=10.0.0.10
web02 ansible_host=10.0.0.11
Group variables:
[webservers:vars]
http_port=80
So:
webservers
  |
  +-- web01
  +-- web02
  +-- web03
     |
     +-- common group vars
Large environment lo static inventory badulu dynamic inventory use cheyyachu.
Example:
AWS API
  ↓
EC2 instances
  ↓
Dynamic Inventory
  ↓
Ansible

---

8. Ad-hoc commands
Ansible lo smallest automation unit laga ad-hoc commands use cheyyachu.
Example:
ansible webservers -m ansible.builtin.ping
Meaning:
webservers group
    ↓
ping module
    ↓
check connectivity
Another example:
ansible webservers -m ansible.builtin.command -a "uptime"
Meaning:
webservers
  ↓
execute uptime
But ad-hoc commands repeated automation kosam kaadu.
Repeated work kosam:
Playbook.

---

9. Playbook — Ansible heart

Playbook = automation blueprint.
Example:

```
---
- name: Configure web servers
  hosts: webservers
  become: true

  tasks:

    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
```

Structure:

```
Playbook
  |
  +-- Play
      |
      +-- hosts
      +-- vars
      +-- tasks
      +-- handlers
      +-- roles
```

Official docs playbooks ni repeatable/reusable configuration and deployment system ga describe chestayi. **A** Ansible Documentation

---

10. Play
One play basically:
Ee hosts meeda ee tasks apply cheyyali?
Example:

```
- name: Configure web servers
  hosts: webservers
```

Another play:

```
- name: Configure databases
  hosts: dbservers
```

One playbook lo multiple plays undachu:

```
site.yml
  |
  +-- Play 1 -> webservers
  |
  +-- Play 2 -> dbservers
  |
  +-- Play 3 -> monitoring
```

Official execution model lo play hosts ni tasks ki map chestundi. **A** Ansible Documentation

---

11. Task
Task = one operation.

```
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
```

Task components:

```
Task
 |
 +-- name
 |
 +-- module
 |
 +-- module arguments
 |
```

+-- optional:
    when
    loop
    register
    notify
    become
    tags

---

12. Module
Module = actual work perform chese component.
Examples:
package management
file management
user management
service management
copy files
template files
cloud operations
network operations
Example:
ansible.builtin.user:
means user management.
ansible.builtin.service:
means service management.
ansible.builtin.copy:
means file copy.
Official docs modules ni task-defined action ni accomplish cheyyadaniki execute/copy chese code/binaries ga describe chestayi. **A**


Ansible Documentation

---

13. FQCN — important modern concept
Modern Ansible playbooks lo:
ansible.builtin.copy
instead of simply:
copy
use cheyyadam recommended.
FQCN:
Fully Qualified Collection Name
Format:
namespace.collection.module
Example:
ansible.builtin.copy
Meaning:
ansible
 |
 +-- builtin
     |
     +-- copy
Enduku?
Different collections same module name kaligi undachu.


FQCN ambiguity reduce chestundi. Current playbook docs kuda FQCN use cheyyamani recommend chestunnayi. **A**
Ansible Documentation

---

14. Idempotency
Ansible lo **most important concept**.
Suppose:
state: present
First run:
nginx absent
  ↓
install
  ↓
changed
Second run:
nginx already present
  ↓

nothing to change
↓
ok
So:
Desired state
↓
Current state compare
↓
Difference?
/ \
YES NO
| |
change ok
Ansible same playbook repeated ga run chesina desired state already reached unte unnecessary modifications avoid cheyyadaniki designed. **A** Ansible Documentation

---

15. changed**,** ok**,** failed**,** unreachable
Ansible output ni ardham cheskovali.
ok
Task successful and no change required.
changed
Task successful and system state changed.
failed
Task execute ayyindi but failed.
unreachable
Ansible host ni reach/connect avvalekapoyindi.
Example:
web01 ok=5 changed=2 failed=0
web02 ok=4 changed=3 failed=0
web03 unreachable
Production troubleshooting lo idi daily use avtundi.

---

16. Variables
Variables are used to represent differences between machines/environments. Official docs variables ni inventory, playbooks, roles, reusable files, command line etc. lo define cheyyachu ani explain chestayi. **A** Ansible Documentation
Example:
app_port: 8080
app_name: myapp
Then:
- name: Configure app
ansible.builtin.template:
src: app.conf.j2
dest: /etc/myapp/app.conf
Template:
port={{ app_port }}
name={{ app_name }}

---

17. Variable precedence
Idi **advanced but very important**.
Same variable different places lo define chesthe:
defaults
inventory
group vars
host vars
play vars
role vars
task vars
extra vars
Which value win avutundi?

Ansible has a defined variable precedence system. Extra vars (-e) are at the highest precedence. **A** Ansible Documentation
Example:

ansible-playbook site.yml -e "app_port=9000"
This can override lower-precedence definitions.
Interview lo:
"What is variable precedence in Ansible?"
ani ask chesthe ee topic explain cheyyagalali.

---

18. Facts
Ansible remote host gurinchi information gather cheyyagaladu.
Example:
OS
distribution
version
architecture
CPU
memory
interfaces
IP addresses
hostname
Concept:
Server
↓
Gather facts
↓
ansible_facts
↓
Use in playbook
Example:
when: ansible_facts.os_family == "Debian"
Then:
Debian → apt
RedHat → dnf

---

19. Magic variables
Ansible lo automatically available konni special variables untayi.
Examples conceptually:
hostvars
groups
inventory_hostname
ansible_facts
play_hosts
Example:
groups['webservers']
means webservers group hosts.
Ivi multi-host orchestration lo useful.

---

20. Loops
Same operation multiple items ki apply cheyyali ante loops.

```
- name: Install packages
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - curl
    - unzip
```

Execution:
git
↓
curl
↓
unzip
Same task.

---

21. Conditions
Programming lo:
if
Ansible lo:
when:
Example:

```
- name: Install nginx
  ansible.builtin.apt:
```

```
  name: nginx
  state: present
 when: ansible_facts.os_family == "Debian"
```

Meaning:

```
OS = Debian?
  |
 YES → execute
  |
 NO → skip
```

---

22. Register

Task output later use cheyyali ante:

```
register: result
```

Example:

```
- name: Check service
 ansible.builtin.command: systemctl is-active nginx
 register: nginx_status
```

Then:

```
- name: Show result
 ansible.builtin.debug:
  var: nginx_status
```

Flow:

```
Task
↓
Output
↓
register
↓
Variable
↓
Later logic
```

---

23. set_fact

Playbook execution madhyalo new variable create cheyyachu.

```
- name: Set application version
 ansible.builtin.set_fact:
  app_version: "2.5"
```

Then:

```
{{ app_version }}
```

use cheyyachu.

---

24. Jinja2 templating

Ansible automation lo **Jinja2** huge topic.

Template:

```
server {
  listen {{ app_port }};
  server_name {{ server_name }};
}
```

Variables:

```
app_port: 8080
server_name: example.com
```

Rendered result:

```
server {
  listen 8080;
  server_name example.com;
}
```

So:

```
Template
  +
Variables
  ↓
Rendered configuration
```

---

25. Jinja2 filters

Data transform cheyyadaniki filters.

Concept examples:

```
{{ name | lower }}
{{ name | upper }}
```

List manipulation, formatting, defaults, transformations etc. kosam filters chala useful.

Advanced Ansible lo Jinja2 almost unavoidable.

---

26. Tests
Jinja expressions lo values conditions/test cheyyadaniki tests use cheyyachu.
Concept:
is defined
is not defined
is string
is number
Example:
{% if app_port is defined %}
...
{% endif %}

---

27. Handlers
Handler = **change jarigithe trigger ayye special task**.
Example:
```
- name: Copy nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx
```
Handler:
handlers:

```
  - name: Restart nginx
    ansible.builtin.service:
      name: nginx
      state: restarted
```
Flow:
```
Config task
    |
    +-- changed?
        |
       YES
        |
      notify
        |
        v
   Restart nginx
```



Handler only notified when relevant task reports changed. **A** Ansible Documentation

---

28. Why handlers?
Without handlers:
Copy config
Restart nginx

Copy config
Restart nginx

Copy config
Restart nginx
Every time restart.
Handlers:
Config changed → restart
Config unchanged → don't restart
Better production behavior.

---

29. become
Linux administration lo root privileges required.
Example:
become: true
Means privilege escalation.
Concept:
SSH user
  ↓
become
  ↓
sudo/root
  ↓

privileged operation
Example package installation.

---

30. Tags
Large playbook:
100 tasks
But only deployment tasks run cheyyali.
Tags:
tags:
 - deploy
Then:
ansible-playbook site.yml --tags deploy
Selective execution.

---

31. Blocks
Related tasks ni group cheyyachu.
block:
 - task1
 - task2
 - task3
More importantly error handling:
block
 ↓
try these tasks

rescue
 ↓
if failed, recover

always
 ↓
run cleanup
Programming try/catch/finally concept laga.

---

32. Error handling
Possible controls:
failed_when
changed_when
ignore_errors
block/rescue/always
any_errors_fatal
ignore_unreachable
Example:
failed_when: result.rc != 0
Meaning custom failure condition.

---

33. changed_when
Ansible task change status ni custom-ga determine cheyyachu.
Example:
changed_when: false
Meaning:
Task execute ayina, state change report cheyyaku.
Automation output clean ga maintain cheyyadaniki useful.

---

34. Includes vs Imports
Large automation ni reusable files ga split cheyyachu.
Conceptually:
main.yml
 |
 +-- task file
 +-- another task file
include_* generally dynamic inclusion.
import_* generally static/preprocessed inclusion.
Advanced playbook architecture lo distinction important.

---

35. Roles
Project grow ayyaka:
site.yml
500 lines:
😵
Instead:

roles/
 nginx/
 java/
 mysql/
 app/
 monitoring/
Role structure:
nginx/
├── tasks/
│ └── main.yml
├── handlers/
│ └── main.yml
├── templates/
├── files/
├── vars/
├── defaults/
├── meta/
└── README.md
Role reusable automation unit.
Official docs role ni tasks, handlers, variables, plugins, templates, files lanti reusable content package laga describe chestayi. **A**


Ansible Documentation

---

36. defaults **vs** vars
Role lo:
defaults/
vars/
rendu untayi.
General rule:
defaults
 ↓
easy to override

vars
 ↓
higher precedence
Example:
defaults/main.yml
app_port: 8080
User override cheyyachu.

---

37. Collections
Modern Ansible ecosystem lo **collections** major concept.
Collection can contain:
modules
plugins
roles
playbooks

Official docs kuda collection ni Ansible content distribution format ani describe chestayi. **A** Ansible Documentation
Concept:
Collection
 |
 +-- Modules
 +-- Plugins
 +-- Roles
 +-- Playbooks
Examples of ecosystems:
cloud collections
network collections
database collections
vendor collections

---

38. Ansible Galaxy
Galaxy = reusable Ansible content discover/share/install cheyyadaniki ecosystem.
Concept:
You

|
v
Galaxy
|
+-- Find role
+-- Find collection
+-- Install
|
v
Your project
But blindly community role install chesi production lo use cheyyakudadhu.
Security, maintenance, version compatibility check cheyyali.

---

39. Vault
Secrets plain text lo:
db_password: supersecret
petti Git lo commit cheyyadam dangerous.
Ansible Vault encrypted content kosam.
Concept:
secret
↓
encrypt
↓
vault file
↓
Git
Runtime:
Vault
↓
decrypt
↓
Ansible
↓
target
Important distinction:
Vault encryption helps protect stored secret content, but it doesn't automatically solve all secret-management problems.
Enterprise lo external secret managers kuda use chestaru.

---

40. Dynamic Inventory
Static:
[web]
web01
web02
Cloud:
EC2 instances constantly changing
Dynamic inventory:
Cloud API
↓
discover hosts
↓
Ansible inventory
Useful for auto-scaling environments.

---

41. Execution model — VERY important
Default behavior ni understand cheyyi.
Suppose:
3 servers
3 tasks
Default linear strategy lo roughly:
Task 1
├— server1
├— server2
└— server3

Task 2
├— server1
├— server2
└— server3

Task 3
├— server1

├— server2
└— server3
Ansible default strategy linear, and current docs explain that each task runs across affected hosts before moving to the next task;



default forks setting is 5 unless changed. **A** Ansible Documentation+1

---

42. Forks
Parallel workers.
Default:
forks = 5
Concept:
100 servers
|
v
parallel workers
|
v
execute batches



Can configure higher if control node/resources/network can handle it. **A** Ansible Documentation

---

43. Serial
Production deployment lo all servers simultaneously update cheyyakunda:
serial: 10
Concept:
100 servers

Batch 1 -> 10
Batch 2 -> 10
Batch 3 -> 10
...



This supports rolling deployments. **A** Ansible Documentation

---

44. Throttle
Specific task ki parallelism limit cheyyachu.
Example concept:
throttle: 1
Meaning that task ni one worker at a time limit cheyyachu.



API rate limit or CPU-heavy operation lanti cases lo useful. **A** Ansible Documentation

---

45. run_once
Task only one host meeda run cheyyali:
run_once: true
Example:
10 web servers
|
+-- DB migration
|
+-- run once
But which host executes and delegation strategy carefully design cheyyali.

---

46. Delegation
Task target host badulu another host/controller/local machine meeda execute cheyyachu.
Example:
Web server
|
| deploy
|
+----> Load balancer
Or:
Remote host

|
+---- delegate task --> localhost
Multi-tier orchestration lo useful.

---

47. Async / Polling
Long task:
backup
database operation
long deployment
Immediate wait cheyyakunda asynchronous job start cheyyachu.
Concept:
start job
↓
job id
↓
poll status
↓
complete

Current playbook docs async actions/polling ni dedicated execution topic ga cover chestayi. **A** Ansible Documentation

---

48. Strategy plugins
Default:
linear
Alternative:
free
free strategy lo host task execution faster ga independently proceed avvachu, rather than waiting for all hosts at each task boundary.

A Ansible Documentation
So:
linear:
A task1
B task1
C task1
then task2

free:
A task1 → task2 → task3...
B task1 → ...
Exact behavior task dependencies/host state batti understand cheyyali.

---

49. Plugins
Modules ≠ plugins.
Module
Actual operation.
Plugin
Ansible behavior extend/change cheyyadaniki.
Important plugin types:
Connection
Lookup
Filter
Callback
Inventory
Strategy
Become

Official docs plugins core capabilities expand chestayi ani explain chestayi. **A** Ansible Documentation

---

50. Connection plugins
How to connect?
SSH
WinRM
local
network-specific connections
Concept:

Ansible
  |
Connection plugin
  |
Target

---

51. Lookup plugins
External/local source nundi values retrieve cheyyadaniki.
Concept:
Ansible
  |
Lookup
  |
file / environment / external source
Important: Lookup execution context and security considerations carefully understand cheyyali.

---

52. Filter plugins
Data transformation.
{{ value | some_filter }}
Custom filters kuda create cheyyachu.

---

53. Callback plugins
Ansible output ni customize cheyyadaniki.
Normal CLI output:
TASK [...]
changed: [...]
ok: [...]
Callback plugins output/logging behavior ni modify cheyyachu.
Enterprise reporting/integration lo useful.

---

54. Custom modules
Built-in module saripokapothe custom module create cheyyachu.
Concept:
Your requirement
  ↓
Custom module
  ↓
Ansible task
But custom module write cheyyadam before trying existing collection/module should be considered.

---

55. Cloud automation
Ansible cloud environments tho interact cheyyagaladu.
Example:
AWS
 |
 +-- EC2
 +-- VPC
 +-- Load Balancer
 +-- Security Groups
or Azure/GCP etc.
Typical architecture:
Terraform
  ↓
Create infrastructure

Ansible
  ↓
Configure infrastructure
But this separation is a common pattern, not a strict rule.

---

56. Network automation
Ansible network devices ni kuda manage cheyyagaladu.
Ansible
  |
  +-- Router
  +-- Switch
  +-- Firewall
  +-- Load balancer
Possible tasks:
VLAN
interfaces

routing
ACL
configuration
backup
compliance

Network automation ki specialized collections/modules use chestaru. **A** Ansible Documentation

---

57. Application deployment
Suppose Java application.
Git
↓
Build
↓
Artifact
↓
Ansible
↓
Servers
Ansible:
download artifact
↓
stop service
↓
backup old version
↓
deploy new version
↓
update config
↓
start service
↓
health check

---

58. Rolling deployment
100 servers.
Instead of:
100 → update all
Use:
10 → health check
10 → health check
10 → health check
...
Ansible serial + health checks + handlers/delegation can participate in this pattern. Official docs explicitly cover rolling updates and multi-machine orchestration. **A** Ansible Documentation+1

---

59. Blue/Green deployment
Concept:

```
        Load Balancer
            |
      +-------+-------+
      |               |
    BLUE            GREEN
    v1.0            v2.0
```

New version Green lo deploy.
Test:
Green healthy?
|
YES
↓
Switch traffic
Ansible can automate these steps.

---

60. Zero-downtime deployment
Typical pattern:
Remove server from LB

↓
Deploy
↓
Health check
↓
Add server back
Then next server.
Server1 → deploy → healthy → traffic
Server2 → deploy → healthy → traffic
Server3 → deploy → healthy → traffic
This is where Ansible becomes more than simple package installation.

---

61. CI/CD integration
Typical pipeline:
Developer
↓
Git
↓
CI
↓
Build
↓
Test
↓
Artifact
↓
Ansible
↓
Deploy
↓
Verify
Ansible itself is not a replacement for every CI/CD platform.
Instead:
Jenkins/GitLab/GitHub Actions
+
Ansible
common combination.

---

62. Ansible + Terraform
Very important distinction.
Terraform
Usually infrastructure lifecycle:
VPC
VM
Subnet
Load Balancer
Database
Ansible
Usually machine/application configuration:
packages
users
config
services
deployment
Typical:
Terraform
↓
Create VM
↓
Ansible
↓
Configure VM
↓
Deploy app

---

63. Ansible + Docker
Possible workflow:
Ansible
↓
Install Docker
↓

Configure Docker
↓
Run containers
But if you're operating a large container platform, Kubernetes may take responsibility for application orchestration.

---

64. Ansible + Kubernetes
Possible:
Ansible
↓
Install/configure Kubernetes
Then:
Kubernetes
↓
Deploy/manage containers
Ansible and Kubernetes solve overlapping-but-different layers.

---

65. Testing
Production automation ni direct ga run cheyyadam risky.
Testing ecosystem:
ansible-lint
Molecule
CI
Typical pipeline:
Code
↓
YAML validation
↓
ansible-lint
↓
Molecule/tests
↓
CI
↓
Deploy

---

66. ansible-lint
It checks common problems and best-practice issues.
Concept:
Bad playbook
↓
ansible-lint
↓
warnings/errors
Code quality improve cheyyadaniki.

---

67. Molecule
Roles/playbooks ni test cheyyadaniki commonly used ecosystem tool.
Concept:
Role
↓
Create test environment
↓
Converge
↓
Verify
↓
Destroy
So role development:
write
↓
test
↓
fix
↓
commit

---

68. check mode
Production change cheyyakunda potential changes estimate cheyyadaniki:
ansible-playbook site.yml --check
But **every module perfect dry-run simulation provide chestundi ani assume cheyyakudadhu**. Module support and task behavior matter.

---

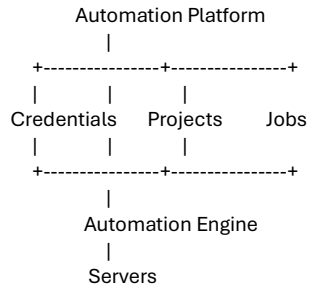
69. diff mode
Config file changes inspect cheyyadaniki:
ansible-playbook site.yml --diff
Useful for:
before
↓
after
comparison.

---

70. Ansible Vault vs Secret Manager
Vault:
Encrypt secrets in Ansible-managed files
Enterprise secret manager:
Centralized secret storage
rotation
access control
audit
dynamic credentials
Examples of external secret-management systems exist, and Ansible can integrate with them through appropriate plugins/collections.
So Vault ≠ universal secret-management solution.

---

71. AWX / Ansible Automation Platform
CLI:
ansible-playbook
works.
But enterprise lo:
100 teams
1000 jobs
many credentials
RBAC
schedules
audit
workflow
job history
requirements vastayi.
Appudu automation platform layer useful.
Concept:



AWX is the upstream open-source project associated with this automation-controller style capability; **Ansible Automation Platform** is Red Hat's supported enterprise offering/ecosystem.

---

72. Execution Environments
Modern enterprise Ansible lo **Execution Environments** important.
Old mental model:
Control machine
|
+-- Python
+-- Ansible
+-- dependencies
Problem:
Developer A → dependency versions
Developer B → different versions
CI → another version
Production → another version
Execution Environment concept:
Container image
|
+-- ansible-core
+-- collections

+-- Python dependencies
+-- system dependencies
Then same environment reproducibly use cheyyachu.

Current docs control node itself container-based execution environments lo run avvachu ani note chestayi. **A** Ansible Documentation

---

73. ansible.cfg
Ansible configuration central place.
Concept:
ansible.cfg
|
+-- inventory
+-- forks
+-- strategy
+-- timeout
+-- roles path
+-- callbacks
+-- SSH settings
Example:
[defaults]
inventory = inventory.ini
forks = 20
But configuration precedence kuda understand cheyyali: defaults, environment, config file, command line etc. Context batti values override avvachu.

---

74. CLI commands you should know
Core commands:
ansible
Ad-hoc commands.
ansible-playbook
Playbooks execute.
ansible-inventory
Inventory inspect/manage.
ansible-vault
Secrets encryption/decryption.
ansible-doc
Module/plugin documentation.
ansible-config
Configuration inspect.
ansible-galaxy
Roles/collections manage.

These are core CLI tools documented by ansible-core. **A** Ansible Documentation

---

75. Troubleshooting flow
Production lo playbook fail aithe panic avvakudadhu.
Flow:
PLAY FAILED
↓
Which host?
↓
Which task?
↓
Which module?
↓
What error?
↓
SSH problem?
↓
Permissions?
↓
Variable?
↓
Template?
↓
Package?

↓
Service?
↓
Network?
Useful:
-v
-vv
-vvv
-vvvv
verbosity increase cheyyachu.
Also:
--check
--diff
debug
register
useful.

---

76. Security best practices
Production Ansible lo:
SSH keys
least privilege
Vault/external secrets
RBAC
Git review
signed/approved content
limited sudo
secure credentials
logging
important.
Especially:
shell:
and:
command:
blind-ga use cheyyakunda appropriate module available unte module use cheyyadam generally better.

---

77. command **vs** shell
Both remote command execution kosam.
command

```
ansible.builtin.command:
  cmd: uptime
```

Simple command execution.
shell

```
ansible.builtin.shell:
  cmd: "cat /tmp/a | grep hello"
```

Shell features like pipes/redirection needed appudu.
But:
If a proper module exists
↓
prefer module
Example:
Bad-ish:
systemctl start nginx
Better:

```
ansible.builtin.service:
  name: nginx
  state: started
```

Why?
Because module desired state/idempotency semantics provide chestundi.

---

78. Idempotency failure example
Bad automation:
shell: echo hello >> /tmp/test
Every run:
hello
hello
hello
hello
So:
Run 1 → change
Run 2 → change
Run 3 → change

Not idempotent.
Better design:
Desired file content
↓
copy/template module
↓
same state
Then repeated runs:
Run 1 → changed
Run 2 → ok
Run 3 → ok

---

79. Production architecture example
Suppose company has:
Internet
|
Load Balancer
|
+---+---+---+
| | | |
Web Web Web
|
App Servers
|
Database
Ansible automation:
Ansible
|
+--------------+--------------+
| | |
Web App DB
| | |
nginx Java/app config
users deployment backup
firewall service users
Then CI/CD:
Developer
↓
Git
↓
CI
↓
Tests
↓
Ansible
↓
Rolling deployment
↓
Health checks
↓
Production

---

80. Enterprise architecture
Large company lo:
Developer
↓
Git repository
↓
Pull Request
↓
Code Review
↓
CI
↓
Lint/Test
↓
Automation Platform
↓
Job Template
↓
Credentials

↓
Execution Environment
↓
Ansible
↓
Dynamic Inventory
↓
Cloud / Servers / Network
This is where Ansible becomes an **automation platform component**, not just a YAML CLI.

---

81. Ansible vs Puppet/Chef
Historically all are configuration-management/automation tools.
High-level distinction:
Ansible
→ agentless-oriented
→ YAML
→ push-style automation

Puppet
→ agent-based model commonly associated
→ declarative language
→ strong configuration-management heritage

Chef
→ Ruby-based DSL
→ agent/client architecture commonly associated
Modern ecosystems evolve, so exact capabilities overlap.

---

82. Ansible vs Bash
Bash:
commands
scripts
shell logic
Ansible:
inventory
modules
desired state
idempotency
multi-host orchestration
variables
roles
structured automation
Bash still has a place.
Ansible doesn't mean:
"Never write shell."
Rather:
"Use the appropriate abstraction."

---

83. Ansible vs Terraform
Again:
Terraform
↓
Infrastructure lifecycle

Ansible
↓
Configuration + deployment + orchestration
Typical:
Terraform
↓
VPC
EC2
LB
Security Groups
↓
Ansible
↓
OS configuration
packages
users
application

---

84. Ansible vs Kubernetes

Ansible
↓
Infrastructure/system/application automation

Kubernetes
↓
Container orchestration/platform
Ansible can even help install/configure Kubernetes.

---

85. The most important mental model

Ansible ni **commands collection** laga think cheyyaku.
Think:
CURRENT STATE
↓
Ansible
↓
DESIRED STATE
Example:
Current:
nginx = absent

Desired:
nginx = present

Ansible:
install nginx
Another:
Current:
nginx config = old

Desired:
nginx config = new

Ansible:
update config
notify restart
Another:
Current:
app = v1

Desired:
app = v2

Ansible:
deploy v2
health check
remove old version

---

86. Complete Ansible learning roadmap

Nuvvu **job-ready + production-ready** ga nerchukovali ante ee sequence best mental roadmap:
LEVEL 1 — FOUNDATION
│
├— Linux
├— SSH
├— YAML
├— Git
└— Basic networking
↓
LEVEL 2 — ANSIBLE CORE
│
├— Installation
├— Inventory
├— Ad-hoc commands
├— Modules
├— Playbooks
├— Plays
└— Tasks
↓

LEVEL 3 — PLAYBOOK DEVELOPMENT
│
├── Variables
├── Facts
├── Loops
├── Conditions
├── Register
├── set_fact
├── Handlers
├── Become
├── Tags
├── Blocks
└── Error handling
↓
LEVEL 4 — DATA & REUSE
│
├── Jinja2
├── Filters
├── Tests
├── Includes
├── Imports
└── Roles
↓
LEVEL 5 — ADVANCED
│
├── Variable precedence
├── Dynamic inventory
├── Delegation
├── run_once
├── serial
├── throttle
├── async
├── strategies
├── plugins
└── custom modules
↓
LEVEL 6 — ECOSYSTEM
│
├── Collections
├── Galaxy
├── Vault
├── ansible.cfg
├── Execution Environments
└── Content management
↓
LEVEL 7 — DEVOPS
│
├── Jenkins/GitLab/GitHub Actions
├── Terraform
├── Docker
├── Kubernetes
├── Cloud
└── CI/CD
↓
LEVEL 8 — PRODUCTION
│
├── Rolling deployments
├── Blue/Green
├── Zero downtime
├── Security
├── Testing
├── ansible-lint
├── Molecule
├── Troubleshooting
└── Performance
↓
LEVEL 9 — ENTERPRISE
│

├── AWX
├── Ansible Automation Platform
├── RBAC
├── Credentials
├── Job Templates
├── Workflows
├── Scheduling
├── Audit
└── Execution Environments

---

87. Finally — "Ansible motham enti?" ante

One diagram lo:

